

## การจัดกระบวนการ ETL ด้วย Apache Airflow และ dbt

Workflow Orchestration with Airflow and dbt

ระยะเวลา 2 ชั่วโมง • ปฏิบัติรายบุคคลหรือคู่

## วัตถุประสงค์

เมื่อสิ้นสุด Lab นี้ นักศึกษาจะสามารถ:

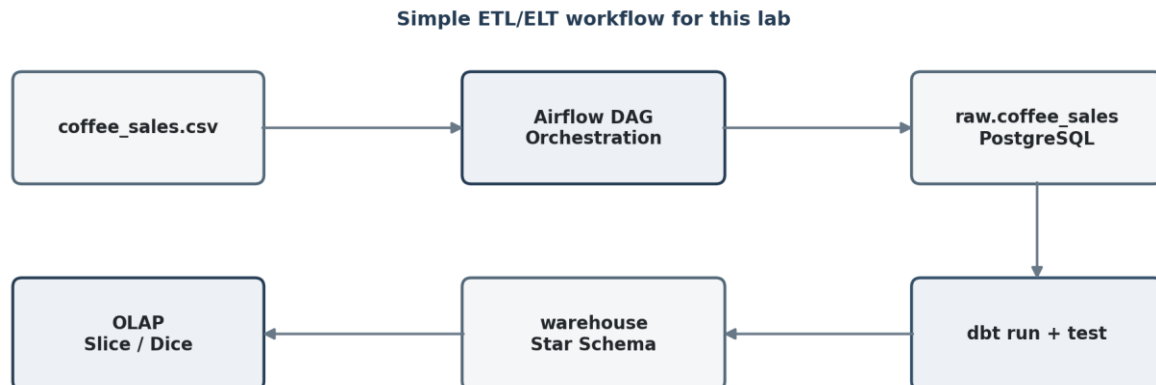
- อธิบายบทบาทที่ต่างกันของ Airflow และ dbt ในกระบวนการนำข้อมูลเข้าสู่ Data Warehouse ได้
- สร้าง DAG เพื่อกำหนดลำดับงาน ตรวจสอบไฟล์ต้นทาง โหลดข้อมูลเข้าสู่ Raw Schema และเรียกใช้ dbt ได้
- ใช้ dbt สร้าง Staging Model, Dimension Table และ Fact Table แบบง่ายใน PostgreSQL ได้
- ตรวจสอบสถานะงานและ Log ผ่าน Airflow UI รวมถึงใช้ dbt test ตรวจสอบคุณภาพข้อมูลพื้นฐานได้
- เขียน SQL เพื่อทำ OLAP Operation แบบ Slice และ Dice จาก Star Schema ได้

## เครื่องมือและไฟล์ที่ใช้

| รายการ               | หน้าที่ใน Lab                                                                                                             |
|----------------------|---------------------------------------------------------------------------------------------------------------------------|
| coffee_sales.csv     | ข้อมูลยอดขาย 3,000 รายการสำหรับโหลดเข้า Raw Schema                                                                        |
| Apache Airflow 3.2.2 | จัดลำดับงาน แสดงสถานะ และเก็บ Log ของ Pipeline                                                                            |
| dbt + dbt-postgres   | แปลงข้อมูลจาก Raw Schema เป็น Star Schema และทดสอบข้อมูล                                                                  |
| PostgreSQL           | เก็บ Airflow metadata ในฐานข้อมูล airflow และเก็บ Raw Data กับ Data Warehouse ของ Lab ในฐานข้อมูล lab10 โดยแยกด้วย Schema |
| pgAdmin / Metabase   | ตรวจสอบตารางและทดลองคำสั่ง SQL สำหรับ Slice และ Dice                                                                      |

## 1. ภาพรวมแนวคิดก่อนลงมือ

Airflow และ dbt ไม่ได้ทำหน้าที่ซ้ำกัน แต่ทำงานร่วมกันเป็น Pipeline เดียว ดังภาพ



ภาพที่ 1 ภาพรวมการไหลของข้อมูลใน Lab

| เครื่องมือ        | รับผิดชอบอะไร                                                         | สิ่งที่เห็นใน Lab                       |
|-------------------|-----------------------------------------------------------------------|-----------------------------------------|
| <b>Airflow</b>    | Orchestration: กำหนดว่าอะไรทำก่อน-หลัง เมื่อใด สำเร็จหรือผิดพลาด      | DAG, Task, Graph, Log, Retry            |
| <b>dbt</b>        | Transformation: ใช้ SQL เปลี่ยนข้อมูล Raw ให้เป็นโมเดลสำหรับวิเคราะห์ | stg_coffee_sales, Dimension, Fact, Test |
| <b>PostgreSQL</b> | Storage: เก็บข้อมูลจริงและผลลัพธ์ของแต่ละขั้นตอน                      | Schema raw และ warehouse                |

**ETL หรือ ELT:** ใน Lab นี้ Airflow อ่านไฟล์และโหลดเข้า raw.coffee\_sales ก่อน จากนั้น dbt จึง Transform ภายใน PostgreSQL ดังนั้นเชิงเทคนิคเป็นรูปแบบ ELT แต่เอกสารใช้คำว่า ETL Pipeline ในความหมายกว้างของกระบวนการนำข้อมูลเข้าสู่คลังข้อมูล

### ลำดับ Task ใน DAG

1. check\_source\_file - ตรวจสอบว่าไฟล์ CSV มีอยู่และมีข้อมูล
2. load\_raw\_csv - ล้างข้อมูลเดิมใน Raw Table แล้วโหลดข้อมูลจาก CSV
3. dbt\_run - สร้าง Staging View, Dimension Tables และ Fact Table
4. dbt\_test - ตรวจสอบ not\_null และ unique ตามที่กำหนดใน schema.yml
5. validate\_warehouse - สรุปจำนวนแถวและยอดขายรวมลงใน Airflow Log

แผนเวลาโดยประมาณ

| กิจกรรม                                          | เวลา    |
|--------------------------------------------------|---------|
| ทำความเข้าใจบทบาท <b>Airflow</b> และ <b>dbt</b>  | 10 นาที |
| เตรียมโครงสร้างโฟลเดอร์และ <b>Docker Compose</b> | 15 นาที |
| สร้าง <b>dbt Project</b> และ <b>Star Schema</b>  | 35 นาที |
| สร้าง <b>Airflow DAG</b>                         | 25 นาที |
| <b>Run, Monitor</b> และตรวจสอบผล                 | 20 นาที |
| ทดลอง <b>Slice</b> และ <b>Dice</b>               | 15 นาที |

## 2. เตรียม Environment

### 2.1 เตรียมไฟล์ข้อมูลต้นทาง

คัดลอก `coffee_sales.csv` ไปไว้ในโฟลเดอร์ `raw_data`

### 2.2 ตรวจสอบโครงสร้างที่ควรได้

```
DWH_Lab/
|-- docker-compose.yaml
|-- dockerfile.airflow
|-- postgresql.conf
|-- dags/
|   |-- coffee_sales_etl_dag.py
|-- raw_data/
|   |-- coffee_sales.csv
|-- dbt/
|   |-- dbt_project.yml
|   |-- models/
|   |   |-- sources.yml
|   |   |-- schema.yml
|   |   |-- staging/
|   |   |   |-- stg_coffee_sales.sql
|   |   |-- marts/
|   |   |   |-- dim_date.sql
|   |   |   |-- dim_product.sql
|   |   |   |-- dim_store.sql
|   |   |   |-- dim_customer.sql
|   |   |   |-- dim_promotion.sql
|   |   |   |-- fct_sales.sql
|   |-- dbt_root/
|   |-- profiles.yml
```

### 2.3 เพิ่ม dbt Project ให้ Airflow มองเห็น

เปิดไฟล์ `docker-compose.yaml` แล้วเพิ่ม Volume สองบรรทัดได้ `x-airflow-common > volumes`

โดยคงรายการเดิมไว้ทั้งหมด

## **docker-compose.yaml** - ส่วนที่ต้องเพิ่มได้ volumes ของ x-airflow-common

```
volumes:
- ${AIRFLOW_PROJ_DIR:-.}/dags:/opt/airflow/dags
- ${AIRFLOW_PROJ_DIR:-.}/logs:/opt/airflow/logs
- ${AIRFLOW_PROJ_DIR:-.}/config:/opt/airflow/config
- ${AIRFLOW_PROJ_DIR:-.}/plugins:/opt/airflow/plugins
- ${AIRFLOW_PROJ_DIR:-.}/raw_data:/home/raw_data
- ${AIRFLOW_PROJ_DIR:-.}/dbt:/opt/airflow/dbt
- ${AIRFLOW_PROJ_DIR:-.}/dbt_root:/opt/airflow/dbt_root
```

**เหตุผล:** Task dbt\_run และ dbt\_test ทำงานภายใน Airflow container จึงต้อง Mount โฟลเดอร์ dbt และ dbt\_root เข้าไปใน container ด้วย

หลังบันทึกไฟล์ หากเคยเริ่ม Airflow Environment ไว้แล้ว ให้สั่ง Docker Compose สร้าง Container ใหม่ เพื่อให้การ Mount Volume ที่เพิ่มมีผล

```
docker compose up -d --force-recreate
docker compose ps
```

**คำแนะนำ:** ไม่ต้องใช้ docker compose down -v เพราะจะลบ Named Volume และข้อมูลเดิม

## **2.4 สร้างฐานข้อมูล lab10**

สร้างฐานข้อมูลแยกสำหรับ Lab 10 เพื่อให้ Raw Data และ Data Warehouse อยู่ในฐานข้อมูล lab10 โดยไม่ปะปนกับฐานข้อมูล airflow ที่ใช้เก็บ Airflow metadata

```
CREATE DATABASE lab10;
```

**หมายเหตุ:** หาก PostgreSQL แจ้งว่า database "lab10" already exists แสดงว่าเคยสร้างไว้แล้ว สามารถใช้ฐานข้อมูลเดิมต่อได้

## **3. สร้าง dbt Project**

### **3.1 สร้าง Profile สำหรับเชื่อมต่อฐานข้อมูล lab10**

#### **dbt\_root/profiles.yml**

```
lab10:
  target: dev
  outputs:
    dev:
      type: postgres
      host: postgres
      port: 5432
      user: dw_user
      password: dw_pass
      dbname: lab10
      schema: warehouse
      threads: 4
```

**จำให้แม่น:** เมื่อเชื่อมต่อจาก container หนึ่งไปอีก container หนึ่ง ต้องใช้ชื่อ service คือ postgres ไม่ใช่ localhost และไม่ใช้ชื่อ container dw\_postgres

### 3.2 กำหนดค่า dbt Project

#### dbt/dbt\_project.yml

```
name: lab10
version: "1.0.0"
config-version: 2

profile: lab10

model-paths: ["models"]
clean-targets: ["target", "dbt_packages"]

models:
  lab10:
    staging:
      +materialized: view
    marts:
      +materialized: table
```

Profile เชื่อมต่อฐานข้อมูล lab10 และกำหนด schema เป้าหมายเป็น warehouse ดังนั้น Model ทั้งหมดจะอยู่ใน warehouse schema โดย Staging เป็น View และ Dimension/Fact เป็น Table

### 3.3 ประกาศ Source Table

#### dbt/models/sources.yml

```
version: 2

sources:
  - name: raw
    schema: raw
    tables:
      - name: coffee_sales
        columns:
          - name: sale_id
            tests:
              - not_null
              - unique
          - name: sale_date
            tests:
              - not_null
```

### 3.4 สร้าง Staging Model

Raw Table เก็บข้อมูลเป็นข้อความเพื่อให้การ Load ง่าย ส่วน Staging Model

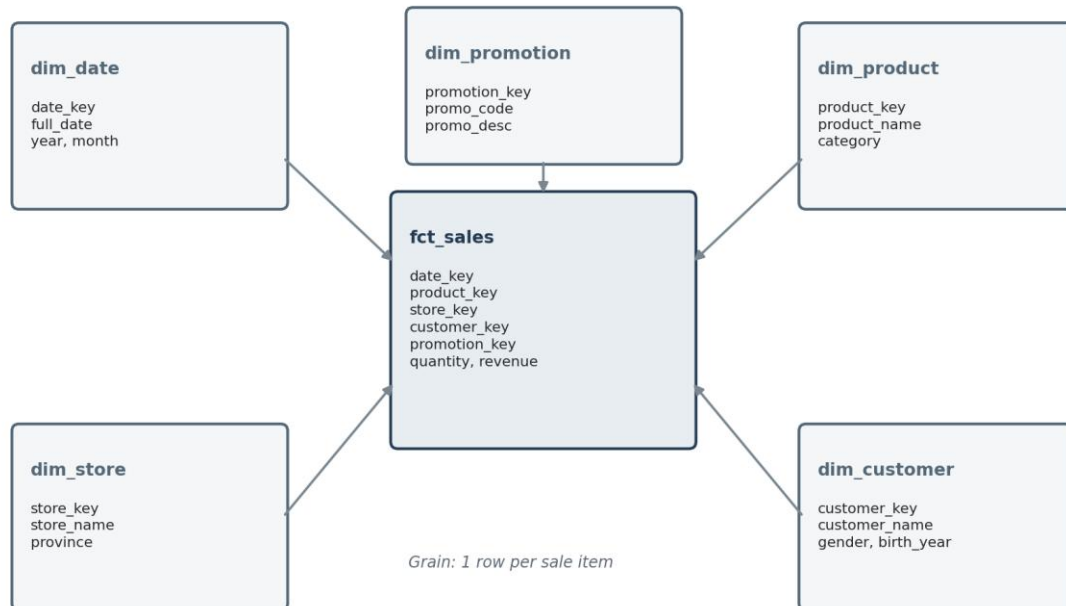
ทำหน้าที่แปลงชนิดข้อมูลและเลือกเฉพาะคอลัมน์ที่ต้องใช้

#### dbt/models/staging/stg\_coffee\_sales.sql

```
select
  sale_id::integer as sale_id,
  invoice_number,
  sale_date::date as sale_date,
  customer_code,
  customer_name,
  gender,
  birth_year::integer as birth_year,
  product_code,
  product_name,
  lower(trim(category)) as category,
  size,
  unit_price::numeric(10, 2) as unit_price,
  quantity::integer as quantity,
  revenue::numeric(12, 2) as revenue,
  store_code,
  store_name,
  province,
  nullif(trim(promo_code), '') as promo_code,
  nullif(trim(promo_desc), '') as promo_desc,
  points_redeemed::integer as points_redeemed
from {{ source('raw', 'coffee_sales') }}
```

## 4. สร้าง Star Schema ด้วย dbt

เพื่อให้สอดคล้องกับ Star Schema ใน Lab ก่อนหน้า Lab นี้ใช้ 5 Dimension และ 1 Fact โดยกำหนด Grain ของ Fact Table เป็น 1 แถวต่อรายการขาย และใช้ md5() สร้าง Surrogate Key ที่คงที่จาก Business Key



ภาพที่ 2 Star Schema แบบย่อที่สร้างด้วย dbt

### 4.1 Dimension สินค้า

#### dbt/models/marts/dim\_product.sql

```

with products as (
  select distinct
    product_code,
    product_name,
    category,
    size,
    unit_price
  from {{ ref('stg_coffee_sales') }}
)

select
  md5(product_code) as product_key,
  product_code,
  product_name,
  category,
  size,
  unit_price
from products
  
```

## 4.2 Dimension ร้านค้า

### dbt/models/marts/dim\_store.sql

```
with stores as (
  select distinct
    store_code,
    store_name,
    province
  from {{ ref('stg_coffee_sales') }}
)

select
  md5(store_code) as store_key,
  store_code,
  store_name,
  province
from stores
```

## 4.3 Dimension ลูกค้า

### dbt/models/marts/dim\_customer.sql

```
with customers as (
  select distinct
    customer_code,
    customer_name,
    gender,
    birth_year
  from {{ ref('stg_coffee_sales') }}
)

select
  md5(customer_code) as customer_key,
  customer_code,
  customer_name,
  gender,
  birth_year
from customers
```

## 4.4 Dimension โปรโมชั่น

### dbt/models/marts/dim\_promotion.sql

```
with promotions as (
  select distinct
    promo_code,
    promo_desc
  from {{ ref('stg_coffee_sales') }}
  where promo_code is not null
)

select
  md5(promo_code) as promotion_key,
  promo_code,
  promo_desc
from promotions
```

โปรโมชั่นเป็นข้อมูลทางเลือก: dim\_promotion เก็บเฉพาะโปรโมชั่นที่มีรหัส ส่วนรายการขายที่ไม่ได้ใช้โปรโมชั่นยังคงอยู่ใน fct\_sales โดยมี promotion\_key เป็น NULL

## 4.5 Dimension วัน

### dbt/models/marts/dim\_date.sql

```
with dates as (
  select distinct sale_date
  from {{ ref('stg_coffee_sales') }}
)

select
  to_char(sale_date, 'YYYYMMDD')::integer as date_key,
  sale_date as full_date,
  extract(year from sale_date)::integer as year,
  extract(month from sale_date)::integer as month_number,
  to_char(sale_date, 'Mon') as month_name,
  extract(day from sale_date)::integer as day_of_month
from dates
```

## 4.6 Fact ยอดขาย

### dbt/models/marts/fct\_sales.sql

```
select
  md5(s.sale_id::text) as sales_key,
  s.invoice_number,
  d.date_key,
  p.product_key,
  st.store_key,
  c.customer_key,
  pr.promotion_key,
  s.quantity,
  s.unit_price,
  s.revenue,
  s.points_redeemed
from {{ ref('stg_coffee_sales') }} s
join {{ ref('dim_date') }} d
  on s.sale_date = d.full_date
join {{ ref('dim_product') }} p
  on s.product_code = p.product_code
join {{ ref('dim_store') }} st
  on s.store_code = st.store_code
join {{ ref('dim_customer') }} c
  on s.customer_code = c.customer_code
left join {{ ref('dim_promotion') }} pr
  on s.promo_code = pr.promo_code
```

## 4.7 กำหนด Data Tests

### dbt/models/schema.yml

```
version: 2

models:
  - name: dim_date
    columns:
      - {name: date_key, tests: [not_null, unique]}

  - name: dim_product
    columns:
      - {name: product_key, tests: [not_null, unique]}
      - {name: product_code, tests: [not_null, unique]}

  - name: dim_store
    columns:
      - {name: store_key, tests: [not_null, unique]}
      - {name: store_code, tests: [not_null, unique]}

  - name: dim_customer
    columns:
      - {name: customer_key, tests: [not_null, unique]}
      - {name: customer_code, tests: [not_null, unique]}

  - name: dim_promotion
    columns:
```



```

- {name: promotion_key, tests: [not_null, unique]}
- {name: promo_code, tests: [not_null, unique]}

- name: fct_sales
  columns:
    - {name: sales_key, tests: [not_null, unique]}
    - {name: date_key, tests: [not_null]}
    - {name: product_key, tests: [not_null]}
    - {name: store_key, tests: [not_null]}
    - {name: customer_key, tests: [not_null]}
    - {name: quantity, tests: [not_null]}
    - {name: revenue, tests: [not_null]}

```

ขอบเขตของ **Lab: Surrogate Key** ของสินค้า ร้านค้า ลูกค้า โปรโมชั่น และรายการขายสร้างด้วย md5() จาก Business Key จึงได้ค่าเดิมทุกครั้ง Build ใหม่ ต่างจาก row\_number() ในงานจริงควรกำหนดมาตรฐานการจัดรูปแบบ Business Key ให้ชัดเจน และออกแบบการเก็บประวัติด้วย SCD แยกต่างหาก

## 5. สร้าง Airflow DAG

สร้างไฟล์ต่อไปนี้จะโดยคัดลอกโค้ดทั้งหมด DAG จะเป็นผู้เรียก dbt run และ dbt test ตามลำดับ

### dags/coffee\_sales\_etl\_dag.py

```

from pathlib import Path

import psycpg2
import pendulum
from airflow.sdk import dag, task
from airflow.providers.standard.operators.bash import BashOperator

CSV_PATH = Path("/home/raw_data/coffee_sales.csv")

@dag(
    dag_id="coffee_sales_etl",
    description="Load coffee sales and build a star schema with dbt",
    start_date=pendulum.datetime(2026, 1, 1, tz="UTC"),
    schedule=None,
    catchup=False,
    default_args={"retries": 1},
    tags=["lab", "dbt", "warehouse"],
)
def coffee_sales_etl():

    @task
    def check_source_file():
        if not CSV_PATH.exists():
            raise FileNotFoundError(f"Source file not found: {CSV_PATH}")

        with CSV_PATH.open("r", encoding="utf-8-sig") as file:
            row_count = sum(1 for _ in file) - 1

        if row_count <= 0:
            raise ValueError("The CSV file has no data rows")

        print(f"Found {row_count:,} rows in {CSV_PATH}")
        return row_count

    @task
    def load_raw_csv():
        columns = """
            sale_id, invoice_number, sale_date, customer_code,
            customer_name, gender, birth_year, product_code,
            product_name, category, size, unit_price, quantity,
            revenue, store_code, store_name, province, staff_code,
            staff_name, position, promo_code, promo_desc,
            points_redeemed
        """

        create_table_sql = """
            create schema if not exists raw;

```

```

create table if not exists raw.coffee_sales (
    sale_id text,
    invoice_number text,
    sale_date text,
    customer_code text,
    customer_name text,
    gender text,
    birth_year text,
    product_code text,
    product_name text,
    category text,
    size text,
    unit_price text,
    quantity text,
    revenue text,
    store_code text,
    store_name text,
    province text,
    staff_code text,
    staff_name text,
    position text,
    promo_code text,
    promo_desc text,
    points_redeemed text
);
truncate table raw.coffee_sales;
"""

connection = psycopg2.connect(
    host="postgres",
    port=5432,
    dbname="lab10",
    user="dw_user",
    password="dw_pass",
)

try:
    with connection.cursor() as cursor:
        cursor.execute(create_table_sql)
        with CSV_PATH.open("r", encoding="utf-8-sig") as file:
            cursor.copy_expert(
                f"""
                copy raw.coffee_sales ({columns})
                from stdin with (format csv, header true)
                """,
                file,
            )
        cursor.execute("select count(*) from raw.coffee_sales")
        row_count = cursor.fetchone()[0]
        connection.commit()
    finally:
        connection.close()

    print(f"Loaded {row_count:,} rows into raw.coffee_sales")
    return row_count

dbt_run = BashOperator(
    task_id="dbt_run",
    bash_command=(
        "dbt run "
        "--project-dir /opt/airflow/dbt "
        "--profiles-dir /opt/airflow/dbt_root"
    ),
)

dbt_test = BashOperator(
    task_id="dbt_test",
    bash_command=(
        "dbt test "
        "--project-dir /opt/airflow/dbt "
        "--profiles-dir /opt/airflow/dbt_root"
    ),
)

@task
def validate_warehouse():

```

```

connection = psycopg2.connect(
    host="postgres",
    port=5432,
    dbname="lab10",
    user="dw_user",
    password="dw_pass",
)
try:
    with connection.cursor() as cursor:
        cursor.execute("""
            select
                count(*) as fact_rows,
                sum(quantity) as total_quantity,
                sum(revenue) as total_revenue
            from warehouse.fct_sales
        """)
        result = cursor.fetchone()
finally:
    connection.close()

print(
    "Warehouse summary: "
    f"rows={result[0]:,}, "
    f"quantity={result[1]:,}, "
    f"revenue={result[2]:,.2f}"
)

source_ok = check_source_file()
raw_loaded = load_raw_csv()
warehouse_ok = validate_warehouse()

source_ok >> raw_loaded >> dbt_run >> dbt_test >> warehouse_ok

coffee_sales_etl()

```

**จุดสังเกต:** เครื่องหมาย >> ใช้กำหนด Dependency ของ Task ไม่ได้หมายความว่า Airflow ส่งข้อมูลทุกแถวจาก Task หนึ่งไปอีก Task หนึ่ง ข้อมูลจริงอยู่ในไฟล์และ PostgreSQL ส่วน XCom ใช้เพียงค่าขนาดเล็ก เช่น row\_count

## 6. Run และติดตามผลใน Airflow

1. เปิด <http://localhost:28080> แล้ว Login ด้วย airflow / airflow
2. ค้นหา DAG ชื่อ coffee\_sales\_etl
3. เปิดสวิตช์ Unpause เนื่องจาก Environment กำหนด DAGS\_ARE\_PAUSED\_AT\_CREATION เป็น true
4. กด Trigger DAG และรอให้ Task ทั้งหมดเป็นสีเขียว
5. เปิด Graph View เพื่อดู Dependency แล้วคลิกแต่ละ Task เพื่อเปิด Log

สิ่งที่ควรเห็น

```

check_source_file
|
v
load_raw_csv
|
v
dbt_run
|
v
dbt_test
|
v
validate_warehouse

```

## ผลตรวจสอบที่คาดหวัง

| ตาราง / ค่า                    | ผลที่คาดหวัง |
|--------------------------------|--------------|
| <b>raw.coffee_sales</b>        | 3,000 แถว    |
| <b>warehouse.dim_product</b>   | 5 แถว        |
| <b>warehouse.dim_store</b>     | 3 แถว        |
| <b>warehouse.dim_customer</b>  | 2,745 แถว    |
| <b>warehouse.dim_promotion</b> | 2 แถว        |
| <b>warehouse.dim_date</b>      | 31 แถว       |
| <b>warehouse.fct_sales</b>     | 3,000 แถว    |
| <b>sum(quantity)</b>           | 6,023        |
| <b>sum(revenue)</b>            | 320,240.00   |

## ตรวจสอบด้วย SQL

```
select 'raw' as object_name, count(*) as row_count
from raw.coffee_sales
union all
select 'dim_product', count(*) from warehouse.dim_product
union all
select 'dim_store', count(*) from warehouse.dim_store
union all
select 'dim_customer', count(*) from warehouse.dim_customer
union all
select 'dim_promotion', count(*) from warehouse.dim_promotion
union all
select 'dim_date', count(*) from warehouse.dim_date
union all
select 'fct_sales', count(*) from warehouse.fct_sales;
```

ทดลอง **Run** ซ้ำ: Trigger DAG อีกรอบ จำนวนแถวควรยังคงเท่าเดิม เพราะ load\_raw\_csv ใช้ TRUNCATE

เพื่อล้างข้อมูลเดิมแล้วโหลดใหม่ และ dbt สร้าง Table ตามผลลัพธ์ล่าสุด กระบวนการนี้เป็น Full Refresh แบบง่าย ไม่ใช่ Incremental Load

## 7. OLAP Operation: Slice และ Dice

ใช้ SQL Editor ใน pgAdmin หรือ Native Query ใน Metabase โดยเชื่อมต่อดูฐานข้อมูล lab10

### 7.1 Slice: เลือกค่าหนึ่งจาก Dimension หนึ่ง

โจทย์: เลือกเฉพาะยอดขายของจังหวัด Bangkok แล้วสรุปตามประเภทสินค้า การกำหนด province เพียงค่าเดียวเปรียบเสมือนการตัด Cube ออกมาเป็น Slice หนึ่ง

```
select
  p.category,
  sum(f.quantity) as total_quantity,
  sum(f.revenue) as total_revenue
from warehouse.fct_sales f
join warehouse.dim_product p
  on f.product_key = p.product_key
join warehouse.dim_store s
  on f.store_key = s.store_key
where s.province = 'Bangkok'
group by p.category
order by p.category;
```

| category | total_quantity | total_revenue |
|----------|----------------|---------------|
| bakery   | 446            | 19,291.50     |
| coffee   | 1,189          | 67,085.00     |
| tea      | 389            | 20,735.00     |

7.2 Dice: เลือกช่วงหรือชุดค่าจากหลาย Dimension

โจทย์: เลือกเฉพาะ Bangkok และ Chiang Mai เฉพาะ coffee และ tea และเฉพาะวันที่ 1-15 กรกฎาคม 2024  
การกรองพร้อมกันหลาย Dimension ทำให้ได้ Sub-cube ขนาดเล็กลง

```
select
  s.province,
  p.category,
  sum(f.quantity) as total_quantity,
  sum(f.revenue) as total_revenue
from warehouse.fct_sales f
join warehouse.dim_date d
  on f.date_key = d.date_key
join warehouse.dim_product p
  on f.product_key = p.product_key
join warehouse.dim_store s
  on f.store_key = s.store_key
where s.province in ('Bangkok', 'Chiang Mai')
  and p.category in ('coffee', 'tea')
  and d.full_date between date '2024-07-01' and date '2024-07-15'
group by s.province, p.category
order by s.province, p.category;
```

| province   | category | total_quantity | total_revenue |
|------------|----------|----------------|---------------|
| Bangkok    | coffee   | 576            | 32,645.00     |
| Bangkok    | tea      | 216            | 11,440.00     |
| Chiang Mai | coffee   | 579            | 32,716.00     |
| Chiang Mai | tea      | 209            | 11,137.50     |

สรุปความแตกต่าง

| Operation | ลักษณะการกรอง                         | ตัวอย่างใน Lab                          |
|-----------|---------------------------------------|-----------------------------------------|
| Slice     | กำหนดค่าเดียวใน Dimension หนึ่ง       | province = 'Bangkok'                    |
| Dice      | กำหนดหลายค่า/หลายช่วงในหลาย Dimension | 2 จังหวัด + 2 ประเภทสินค้า + ช่วงวันที่ |

## รายการส่งงาน

- ภาพหน้าจอ Graph View ของ Airflow ที่ Task ทั้ง 5 สำเร็จ

## 9. การแก้ปัญหาเบื้องต้น

| อาการ                   | จุดที่ควรตรวจสอบ                                                                             |
|-------------------------|----------------------------------------------------------------------------------------------|
| DAG ไม่ปรากฏ            | ชื่อไฟล์ต้องลงท้าย .py ตรวจสอบ Import Error และ Log ของ airflow-scheduler                    |
| dbt: command not found  | Build image ใหม่หลังตรวจสอบว่า dockerfile.airflow มีคำสั่ง pip install dbt-core dbt-postgres |
| Connection refused      | ภายใน container ต้องใช้ host: postgres และ port: 5432 ไม่ใช่ localhost:25432                 |
| ไม่พบ coffee_sales.csv  | ตรวจสอบว่าไฟล์อยู่ raw_data/ และ Volume ชี้ไป /home/raw_data                                 |
| ไม่พบ profiles.yml      | ตรวจสอบ Volume dbt_root และใช้ --profiles-dir /opt/airflow/dbt_root                          |
| ตารางไปอยู่ schema อื่น | ตรวจสอบ schema: warehouse ใน profiles.yml และตรวจสอบว่า DAG ใช้ Project/Profile ที่ถูกต้อง   |
| dbt test ล้มเหลว        | เปิด Log ของ Task dbt_test แล้วดูชื่อ Test ที่ Fail ก่อนตรวจสอบข้อมูลใน Raw และ Dimension    |

### คำสั่งช่วยตรวจสอบ

```
# ดูสถานะ container
docker compose ps

# ดู Log ของ scheduler
docker compose logs airflow-scheduler

# ดู DAG import errors
docker compose exec airflow-scheduler \
  airflow dags list-import-errors

# ตรวจสอบการเชื่อมต่อ dbt
docker compose exec airflow-scheduler dbt debug \
  --project-dir /opt/airflow/dbt \
  --profiles-dir /opt/airflow/dbt_root
```

## สรุปท้าย Lab

Pipeline ที่สร้างใน Lab นี้แยกความรับผิดชอบชัดเจน: Airflow ดูแลลำดับและสถานะของงาน dbt ดูแลการแปลงและทดสอบข้อมูล PostgreSQL เก็บ Raw และ Warehouse Models และผู้ใช้วิเคราะห์ข้อมูลด้วย OLAP Slice/Dice จาก Star Schema ผลลัพธ์จึงครอบคลุมตั้งแต่ไฟล์ต้นทางจนถึงคำตอบเชิงวิเคราะห์ในกระบวนการเดียว